

TerminPilot – Notfall- und Eskalationsplan

1. Zweck des Plans

TerminPilot verarbeitet Verfügbarkeiten aus externen Kalendersystemen. Dadurch können zum Beispiel Probleme mit Kalender-APIs, Synchronisation, Berechtigungen oder dem Zeitplan entstehen.

Der Notfall- und Eskalationsplan soll sicherstellen, dass bei Problemen nachvollziehbar gehandelt wird. Er beantwortet folgende Fragen:

- Was ist passiert?
- Wie kritisch ist das Problem?
- Wer übernimmt welche Aufgabe?
- Welche Sofortmaßnahme ist notwendig?
- Muss der Zeitplan oder der Projektumfang angepasst werden?
- Welche Erkenntnis ergibt sich für die weitere Projektarbeit?

2. Zuständigkeiten

Bereich	Zuständig	Aufgabe im Notfall
Projektkoordination / Product Owner	Lotta Kristin Scholze	Bewertet Auswirkungen auf Zeitplan, Scope und Stakeholder; stimmt Änderungen mit dem Team ab
Technik / Architektur	Leon Reusch	Prüft technische Ursachen und koordiniert technische Lösungsentscheidungen
Backend / API-Integration	Jannis Dickel und Leon Huttarsch	Analysieren Fehler in Kalenderanbindung, Synchronisation oder Datenverarbeitung; setzen technische Fixes um
Qualitätssicherung	Marlene Glaab	Reproduziert Fehler, dokumentiert sie und prüft den Fix anhand von Testfällen
Team	Alle Projektmitglieder	Wird bei größeren Problemen einbezogen und priorisiert gemeinsam Aufgaben, Aufwand oder Scope

3. Einstufung von Problemen

Stufe	Bedeutung	Beispiel	Reaktion
Rot – kritisch	Kernfunktion ist nicht nutzbar oder es besteht ein Datenschutzverdacht	API-Token wurde veröffentlicht; fremde Kalenderdaten sind sichtbar	Sofort informieren, betroffene Funktion gegebenenfalls sperren, Ursache untersuchen und Vorfall dokumentieren
Gelb – wichtig	Zentrale Funktion ist eingeschränkt oder ein Meilenstein gefährdet	Synchronisation mit Google-Kalendern ist nicht zuverlässig	Noch am selben Arbeitstag analysieren; Lösung und mögliche Plananpassung festlegen
Blau – normal	Fehler ist vorhanden, die Kernfunktion läuft aber weiter	Filter funktioniert nur in einem Sonderfall nicht	Als GitHub-Issue erfassen und im nächsten Weekly bzw. Backlog priorisieren
Grau – klein	Kein direkter Einfluss auf Funktion, Datenschutz oder Zeitplan	Schreibfehler oder kleiner Layoutfehler	Im Backlog sammeln und bei freier Kapazität bearbeiten

4. Standardablauf bei einem Problem

1. Erfassen

- Ein GitHub-Issue dokumentiert Beschreibung, Zeitpunkt, betroffenes System, Screenshot/Log und Schritte zum Nachstellen.

2. Eingrenzen

- Es wird geprüft, ob nur ein Testfall, eine Teilfunktion oder der Kern-Use-Case betroffen ist. Zusätzlich wird geklärt, ob personenbezogene Daten betroffen sein könnten.

3. Absichern

- Falls notwendig, wird die betroffene Funktion in der Testumgebung deaktiviert, ein getesteter Stand wiederhergestellt oder ein Zugang/Token gesperrt.

4. Entscheiden

- Der Tech Lead bewertet die technische Lösung. Der Product Owner bewertet Auswirkungen auf Aufwand, Terminplan und Scope.

5. Beheben und testen

- Nach der Implementierung eines Fixes prüft QA sowohl den ursprünglichen Fehlerfall als auch relevante Regressionen.

6. Dokumentieren

- Das GitHub-Issue wird ergänzt oder geschlossen. Je nach Bedeutung werden ChangeLog, Risiko-Log, Testprotokoll, Zeitplan und ADR aktualisiert.

7. Lernen

- Bei größeren Problemen wird die Ursache, die Wirkung und eine Präventionsmaßnahme für spätere Projekte festgehalten.

5. Eskalationsregeln

Auslöser	Vorgehen
Kernfunktion nicht nutzbar	Unmittelbare Teamabsprache; technische Lösung, Testplan und mögliche Zwischenlösung festlegen
Verdacht auf Datenschutzproblem	Betroffene Funktion/Zugriff begrenzen; Datenpannenprotokoll starten; technische und organisatorische Bewertung dokumentieren
Meilenstein mehr als eine Woche gefährdet	Plan-Ist-Abweichung erfassen, Restaufwand neu schätzen, Risikopuffer/Scope prüfen und Entscheidung dokumentieren
Längerer Ausfall eines Teammitglieds	Aufgaben verteilen, Dokumentation prüfen und optionale Funktionen gegebenenfalls verschieben oder streichen
Änderung oder Ausfall einer Kalender-API	API-Changelog und Fehlerbild prüfen; Testfälle anpassen; technische Alternative oder vereinfachten Scope bewerten

6. Mögliche Notfallszenarien

Szenario	Sofortmaßnahme	Mögliche weitere Entscheidung	Nachweis für Abschluss
Kalender-API nicht erreichbar / API-Limit erreicht	Weitere Aufrufe begrenzen; verständliche Fehlermeldung anzeigen; Fehler erfassen	Abfragen bündeln, Wartezeiten bei Wiederholungen einbauen, optionale Funktion verschieben	Integrationstest und Regressionstest erfolgreich; ADR/ChangeLog aktualisiert
Synchronisationsfehler	Betroffene Synchronisation stoppen; Testdaten prüfen; keinen fehlerhaften Kalendereintrag erzeugen	Rollback, Bugfix priorisieren oder Workaround bereitstellen	Fehlerfall und Erfolgsfall im Systemtest bestanden

API-Token oder Secret möglicherweise offengelegt	Token unverzüglich sperrern/rotieren; Fundstelle entfernen; Logs prüfen	Kritischen Incident dokumentieren; Datenschutzbewertung durchführen, wenn Datenzugriff möglich war	Neues Secret aktiv; altes Secret ungültig; Test erfolgreich
Zeitverzug	Restaufwand und Risikopuffer prüfen	Scope reduzieren, Aufgaben umverteilen oder Terminplan anpassen	Aktualisiertes Gantt/Backlog und begründete Entscheidung
Team-Ausfall / Wissenslücke	README, ADRs und bestehende Dokumentation prüfen	Übergabe, Pairing oder Umverteilung; optionale Features verschieben	Neue zuständige Person kann Komponente bauen und testen